13. Go Program for CRUD Operations Using MySQL — From Scratch

https://dev.mysql.com/downloads/installer/

Create a Go Project Create a directory:

- mkdir go-mysql-crud
- cd go-mysql-crud

Initialize a Go module:
go mod init go-mysql-crud

Install MySQL Driver
go get github.com/go-sql-driver/mysql

DB:

```
CREATE DATABASE college;
USE college;
```

/*Create a table:*/

```
CREATE TABLE students (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
age INT NOT NULL,
course VARCHAR(100) NOT NULL
);
```

/*Check the table:*/

```
DESC students;
```

Code:

```
package main
import (
    "database/sql"
    "fmt"

    _ "github.com/go-sql-driver/mysql"
)

var db *sql.DB

func main() {
    // Connect to MySQL
    var err error
    db, err = sql.Open(
```

```
        "mysql",
        "root:Password@tcp(127.0.0.1:3306)/db",
    )
    if err != nil {
        fmt.Println("Error opening database:", err)
        return
    }
    // Check database connection
    err = db.Ping()
    if err != nil {
        fmt.Println("Error connecting to database:", err)
        return
    }
    fmt.Println("Connected to MySQL successfully!")
    defer db.Close()
    for {
        fmt.Println("\n===== STUDENT MANAGEMENT SYSTEM =====")
        fmt.Println("1. Create Student")
        fmt.Println("2. Read Students")
        fmt.Println("3. Update Student")
        fmt.Println("4. Delete Student")
        fmt.Println("5. Exit")
        var choice int
        fmt.Print("Enter your choice: ")
        fmt.Scan(&choice)
        switch choice {
        case 1:
            createStudent()
        case 2:
            readStudents()
        case 3:
            updateStudent()
        case 4:
            deleteStudent()
        case 5:
            fmt.Println("Program terminated.")
            return
        default:
            fmt.Println("Invalid choice!")
        }
    }
}

// CREATE
func createStudent() {
    var name string
```

```
    var age int
    var course string
    fmt.Print("Enter student name: ")
    fmt.Scan(&name)
    fmt.Print("Enter age: ")
    fmt.Scan(&age)
    fmt.Print("Enter course: ")
    fmt.Scan(&course)
    query := `
INSERT INTO students (name, age, course)
VALUES (?, ?, ?)
`
    result, err := db.Exec(query, name, age, course)
    if err != nil {
        fmt.Println("Error inserting student:", err)
        return
    }
    id, err := result.LastInsertId()
    if err != nil {
        fmt.Println("Error getting student ID:", err)
        return
    }
    fmt.Println("Student created successfully!")
    fmt.Println("Student ID:", id)
}

// READ
func readStudents() {
    query := `
SELECT id, name, age, course
FROM students
`
    rows, err := db.Query(query)
    if err != nil {
        fmt.Println("Error reading students:", err)
        return
    }
    defer rows.Close()
    fmt.Println("\nID\tName\tAge\tCourse")
    fmt.Println("--------------------------------")
    for rows.Next() {
        var id int
        var name string
        var age int
        var course string
        err := rows.Scan(
```

```
            &id,
            &name,
            &age,
            &course,
        )
        if err != nil {
            fmt.Println("Error scanning data:", err)
            return
        }
        fmt.Printf(
            "%d\t%s\t%d\t%s\n",
            id,
            name,
            age, course,
        )
    }
    if err := rows.Err(); err != nil {
        fmt.Println("Error reading rows:", err)
    }
}

// UPDATE
func updateStudent() {
    var id int
    var name string
    var age int
    var course string
    fmt.Print("Enter student ID to update: ")
    fmt.Scan(&id)
    fmt.Print("Enter new name: ")
    fmt.Scan(&name)
    fmt.Print("Enter new age: ")
    fmt.Scan(&age)
    fmt.Print("Enter new course: ")
    fmt.Scan(&course)
    query := `
UPDATE students
SET name = ?, age = ?, course = ?
WHERE id = ?
`
    result, err := db.Exec(
        query,
        name,
        age,
        course,
        id,
```

```
    )
    if err != nil {
        fmt.Println("Error updating student:", err)
        return
    }
    rowsAffected, err := result.RowsAffected()
    if err != nil {
        fmt.Println("Error checking update:", err)
        return
    }
    if rowsAffected == 0 {
        fmt.Println("Student not found.")
        return
    }
    fmt.Println("Student updated successfully!")
}

// DELETE
func deleteStudent() {
    var id int
    fmt.Print("Enter student ID to delete: ")
    fmt.Scan(&id)
    query := `
DELETE FROM students
WHERE id = ?
`
    result, err := db.Exec(query, id)
    if err != nil {
        fmt.Println("Error deleting student:", err)
        return
    }
    rowsAffected, err := result.RowsAffected()
    if err != nil {
        fmt.Println("Error checking deletion:", err)
        return
    }
    if rowsAffected == 0 {
        fmt.Println("Student not found.")
        return
    }
    fmt.Println("Student deleted successfully!")
}
```

14. Go Program to create multiple goroutines and implement how the goroutines scheduler behaves with three logical processors for CRUD using MYSQL from scratch

```
CREATE DATABASE college;
USE college;
/*Create a table:*/
CREATE TABLE students (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
age INT NOT NULL,
course VARCHAR(100) NOT NULL
);
/*Check the table:*/
DESC students;
```

Code:

```
package main
import (
   "database/sql"
   "fmt"
   "log"
   "runtime"
   "sync"

   _ "github.com/go-sql-driver/mysql"
)

var db *sql.DB

// CREATE
func createStudent(workerID int, wg *sync.WaitGroup) {
   defer wg.Done()
   name := fmt.Sprintf("Student%d", workerID)
   age := 20 + workerID
   course := fmt.Sprintf("Course%d", workerID)

   query := `INSERT INTO students (name, age, course) VALUES (?, ?, ?)`
   result, err := db.Exec(query, name, age, course)
   if err != nil {
      log.Printf("Worker %d: INSERT error: %v\n", workerID, err)
      return
   }
   id, _ := result.LastInsertId()
   log.Printf("Worker %d: INSERT completed - ID=%d\n", workerID, id)
}
```

```
// READ
func readStudents(workerID int, wg *sync.WaitGroup) {
    defer wg.Done()
    rows, err := db.Query("SELECT id, name, age, course FROM students")
    if err != nil {
        log.Printf("Worker %d: READ error: %v\n", workerID, err)
        return
    }
    defer rows.Close()

    for rows.Next() {
        var id, age int
        var name, course string
        rows.Scan(&id, &name, &age, &course)
        log.Printf("Worker %d: READ -> ID=%d, Name=%s, Age=%d, Course=%s\n",
            workerID, id, name, age, course)
    }
}

// UPDATE
func updateStudent(workerID int, wg *sync.WaitGroup) {
    defer wg.Done()
    query := `UPDATE students SET course=? WHERE id=?`
    _, err := db.Exec(query, fmt.Sprintf("UpdatedCourse%d", workerID), workerID)
    if err != nil {
        log.Printf("Worker %d: UPDATE error: %v\n", workerID, err)
        return
    }
    log.Printf("Worker %d: UPDATE completed\n", workerID)
}

// DELETE
func deleteStudent(workerID int, wg *sync.WaitGroup) {
    defer wg.Done()
    query := `DELETE FROM students WHERE id=?`
    _, err := db.Exec(query, workerID)
    if err != nil {
        log.Printf("Worker %d: DELETE error: %v\n", workerID, err)
        return
    }
    log.Printf("Worker %d: DELETE completed\n", workerID)
}

func main() {
    // Set goroutine scheduler to use 3 logical processors
    runtime.GOMAXPROCS(3)
```

```
    // Connect to MySQL
    var err error
    db, err = sql.Open("mysql", "root:Passwor@tcp(127.0.0.1:3306)/college")
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    // Test connection
    if err := db.Ping(); err != nil {
        log.Fatal("Database connection failed:", err)
    }
    log.Println("Connected to MySQL successfully!")

    // WaitGroup for goroutines
    var wg sync.WaitGroup

    // Spawn multiple goroutines for CRUD
    for i := 1; i <= 5; i++ {
        wg.Add(4)
        go createStudent(i, &wg)
        go readStudents(i, &wg)
        go updateStudent(i, &wg)
        go deleteStudent(i, &wg)
    }

    // Wait for all goroutines to finish
    wg.Wait()
    log.Println("All goroutines completed.")
}
```